LECTURE 06

Introduction to Database Systems

Intermediate SQL

Xiaofan Li

School of Computer Science, Nanjing University

Learning Objectives

01 Use nested queries in `WHERE`, `FROM`, and `SELECT`.

02 Modify data safely with `DELETE`, `INSERT`, `UPDATE`, and `CASE`.

03 Choose join semantics and define, expand, and assess views.

04 Use transactions, constraints, types, indexes, and privileges to govern a database.

Lecture Outline

I. Nested Queries

01 Membership tests
02 SOME and ALL
03 EXISTS and UNIQUE
04 Subqueries as tables and values

II. Data Changes

05 Modifying data

DELETE and INSERT; UPDATE and CASE; queries inside data changes

III. Joins and Views

06 Cross, inner, and outer joins
07 Defining and expanding views
08 Updating and materializing views

IV. Database Control

09 Transactions and constraints
10 Types and indexes
11 Privileges and roles

SECTION 01 OF 11

Membership Tests

Use nested queries to test whether a value or tuple belongs to a query result.

Where a Subquery Can Appear

A subquery is a query inside another SQL statement. Its result can be used as a value, a table, or an input to a condition.

SELECT	FROM	WHERE
one scalar value	a derived table	input to a condition

A **scalar subquery** must return one column and at most one row. One row supplies its value; no rows give NULL; multiple rows cause an error.

```
SELECT (SELECT salary FROM instructor
  WHERE dept_name = 'Finance')
 AS salary;
```

If Finance has salaries 90000 and 80000, this query fails: the outer SELECT expects one value.

Subquery Tests in `WHERE`

Typical predicates ask about a subquery result as a set or relation.

Membership

`IN`, `NOT IN`

Comparison

`SOME`, `ALL`

Cardinality

`EXISTS`

Under SQL three-valued logic, a `NULL` inside these tests can produce `UNKNOWN`.

IN Rephrases an Intersection

Find courses offered in Fall 2009 and Spring 2010.

Schema: `section(course_id, sec_id, semester, year, building, room_number, time_slot_id)`

Correct: match the course across rows

```
SELECT DISTINCT course_id
FROM section
WHERE semester='Fall' AND year=2009
 AND course_id IN (
  SELECT course_id FROM section
  WHERE semester='Spring' AND year=2010
 );
```

`IN` checks whether the Fall row's course also occurs in a Spring row. The two rows share `course_id`.

Incorrect: require both in one row

```
SELECT DISTINCT course_id
FROM section
WHERE semester='Fall'
 AND year=2009
 AND semester='Spring'
 AND year=2010;
```

`AND` tests one row: its semester cannot be both Fall and Spring, nor its year both 2009 and 2010. This query returns no rows.

NOT IN Rephrases a Difference

Find courses offered in Fall 2009 but not in Spring 2010.

Schema: section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

Correct: exclude Spring course IDs

```
SELECT DISTINCT course_id
FROM section
WHERE semester='Fall' AND year=2009
 AND course_id NOT IN (
  SELECT course_id FROM section
  WHERE semester='Spring' AND year=2010
 );
```

Incorrect: exclude Spring on this row

```
SELECT DISTINCT course_id
FROM section
WHERE semester='Fall'
 AND year=2009
 AND NOT (semester='Spring'
      AND year=2010);
```

Counterexample: CS101 has a Fall 2009 row and a Spring 2010 row. The correct query excludes CS101. The wrong query keeps its Fall row: that row is already not a Spring 2010 row. It returns all Fall 2009 courses.

NULL reminder: if the subquery can return NULL, prefer correlated `NOT EXISTS`; `NOT IN` may yield UNKNOWN.

Tuple Membership

Count distinct students who took a section taught by instructor 10101.

`takes(ID, course_id, sec_id, semester, year, grade)` **ID = student ID**
`teaches(ID, course_id, sec_id, semester, year)` **ID = instructor ID**

```
SELECT COUNT(DISTINCT ID)
FROM takes
WHERE (course_id, sec_id, semester, year) IN (
SELECT course_id, sec_id, semester, year
FROM teaches
WHERE ID=10101
);
```

Membership can compare a complete row value, not only one scalar attribute.

SECTION 02 OF 11

SOME and ALL

Compare a value with at least one result or with every result; work through truth values.

SOME Means “At Least One”

Find instructors whose salary exceeds at least one Biology salary.

```
SELECT name FROM instructor
WHERE salary > SOME (
SELECT salary FROM instructor
WHERE dept_name='Biology'
);
```

Compare scalar values

Let x be the outer salary and r the single-column subquery result.

$(x > \text{SOME}(r))$ is TRUE
$\iff \exists t \in r :$
$(x > t[\textit{salary}])$ is TRUE

$t[\textit{salary}]$ is the salary value in row t, not the whole row.

SQL truth rules: TRUE if any comparison is TRUE; FALSE if all are FALSE (also for an empty result); otherwise UNKNOWN. `NULL` comparisons may yield UNKNOWN.
`ANY` and `SOME` are synonyms.

SOME: Worked Truth Values

Let $r = \{0, 5, 6\}$. One witness is enough.

Predicate	Value	Witness or reason
$5 < \text{SOME}(r)$	TRUE	$5 < 6$
$5 = \text{SOME}(r)$	TRUE	$5 = 5$
$5 \neq \text{SOME}(r)$	TRUE	$5 \neq 0$
$5 < \text{SOME}(\{0, 5\})$	FALSE	no value exceeds 5
$5 < \text{SOME}(\emptyset)$	FALSE	no witness exists

Think: Are `<> SOME` and `NOT IN` equivalent?
For $r = \{0, 5, 6\}$, compare `5 <> SOME (SELECT v FROM r)` with `5 NOT IN (SELECT v FROM r)`.

<> SOME vs. NOT IN

Let r(v) contain three rows: 0, 5, and 6. SQL <> means $\neq$.

```
5 <> SOME (SELECT v FROM r)
```

$(5 \neq 0) \vee (5 \neq 5) \vee (5 \neq 6)$
TRUE $\vee$ FALSE $\vee$ TRUE
$= \mathbf{TRUE}$

At least one value differs from 5.

```
5 NOT IN (SELECT v FROM r)
```

$(5 \neq 0) \wedge (5 \neq 5) \wedge (5 \neq 6)$
TRUE $\wedge$ FALSE $\wedge$ TRUE
$= \mathbf{FALSE}$

5 occurs in the result, so NOT IN fails.

They are not equivalent. <> SOME needs one unequal value.
NOT IN requires inequality with every value: it is equivalent to <> ALL.

ALL Means “Every Value”

Find instructors whose salary exceeds every Biology salary.

```
SELECT name FROM instructor
WHERE salary > ALL (
SELECT salary FROM instructor
WHERE dept_name='Biology'
);
```

Compare scalar values

Let x be the outer salary and r the single-column subquery result.

$(x > \texttt{ALL}(r))$ is TRUE
$\iff \forall t \in r :$
$(x > t[\textit{salary}])$ is TRUE

$t[\textit{salary}]$ is the salary value in row t, not the whole row.

SQL truth rules: TRUE if all comparisons are TRUE (also for an empty result); FALSE if any is FALSE; otherwise UNKNOWN. NULL comparisons may yield UNKNOWN.

ALL requires every comparison to be TRUE; SOME requires at least one.

ALL: Worked Truth Values

Every comparison must hold.

Predicate	Value	Reason
$5 < \texttt{ALL}(\{0, 5, 6\})$	FALSE	fails for 0 and 5
$5 < \texttt{ALL}(\{6, 10\})$	TRUE	both exceed 5
$5 = \texttt{ALL}(\{4, 5\})$	FALSE	fails for 4
$5 \neq \texttt{ALL}(\{4, 6\})$	TRUE	both are unequal
$5 < \texttt{ALL}(\emptyset)$	TRUE	no rows to check

Why is the last row TRUE?

It asks: "Is $5 < v$ true for every returned value v?"

An empty result has no values, **so none violates the requirement**. This is called **vacuous truth**.

Empty is not NULL: zero rows make `5 < ALL (...)` TRUE; one row containing `NULL` makes it UNKNOWN.

Reading Quantifier Symbols

UNIVERSAL QUANTIFIER

$$\mathrm{A} \longrightarrow \forall$$

An upside-down capital A.
Read: **“for all”** or “for every.”
Memory aid: A for **All**.

EXISTENTIAL QUANTIFIER

$$\mathrm{E} \longrightarrow \exists$$

A backwards capital E (left–right flip).
Read: **“there exists.”**
Memory aid: E for **Exists**.

$\forall x \in \{6, 10\} : \; x > 5$
Every value in this set is greater than 5.

$\exists x \in \{0, 6\} : \; x > 5$
At least one value is greater than 5: 6.

SECTION 03 OF 11

EXISTS and UNIQUE

Test whether matching rows exist, connect inner and outer queries, and express universal conditions.

`EXISTS` Tests Nonemptiness

NONEMPTY

$$\texttt{EXISTS}(r) \iff r \neq \emptyset$$

EMPTY

$$\texttt{NOT EXISTS}(r) \iff r = \emptyset$$

The selected columns inside an `EXISTS` subquery do not matter; only whether at least one row is produced matters.

Correlated EXISTS

Find courses offered in both Fall 2009 and Spring 2010.

Why correlated? The inner query references `S.course_id` from the current outer row. Its existence test therefore depends on that row and is conceptually repeated for each outer row.

```
SELECT S.course_id
FROM section AS S
WHERE S.semester='Fall' AND S.year=2009
  AND EXISTS (
    SELECT 1 FROM section AS T
    WHERE T.semester='Spring' AND T.year=2010
      AND S.course_id=T.course_id
  );
```

section (relevant columns shown)

course_id	sec_id	semester	year
CS101	1	Fall	2009
CS202	1	Fall	2009
CS101	1	Spring	2010
CS101	2	Spring	2010
CS303	1	Spring	2010

S and T refer to the same table.
`SELECT 1` returns a constant per match.

Logical walkthrough: Keep Fall 2009 rows; test each course against Spring 2010 rows.
`S.course_id` = `CS101`: two matches → `EXISTS` = `TRUE` → output `CS101` once.
`S.course_id` = `CS202`: no matches → `EXISTS` = `FALSE` → discard the row.

This is a conceptual order; the optimizer may use a different plan. Use `DISTINCT` if outer rows repeat a course.

NOT EXISTS Expresses “For All”

Find students who have taken every Biology course.

```
SELECT DISTINCT S.ID, S.name FROM student AS S
WHERE NOT EXISTS (
  SELECT course_id FROM course
  WHERE dept_name='Biology'
  EXCEPT
  SELECT T.course_id FROM takes AS T
  WHERE T.ID=S.ID
);
```

Example data

`course`: the Biology courses are `BIO101` and `BIO102`.

`student` + `takes`:
Alice (ID 1) took `BIO101`, `BIO102`, and `CS101`.

Bob (ID 2) took only `BIO101`.

For each student: all Biology courses `EXCEPT` courses that student took = missing courses.

Student	EXCEPT result	NOT EXISTS	Output (ID, name)
Alice	empty	TRUE	(1, Alice)
Bob	`BIO102`	FALSE	excluded

Schemas (ID identifies a student)

```
student(ID, name, dept_name, tot_cred)
course(course_id, title, dept_name, credits)
takes(ID, course_id, sec_id, semester,
  year, grade)
```

UNIQUE: Testing a Subquery Result

Question: does the subquery return any duplicate rows?

```
UNIQUE (SELECT course_id FROM section)
```

TRUE if every returned course ID occurs at most once; FALSE if any is repeated.

This is a Boolean test of the **current query result**; it does not prevent future inserts.
Here `section.course_id` is NOT NULL. Multiple sections of the same course are allowed.

Think for one minute
For `CS101, CS101, CS202`, what does the predicate return?
How could you perform the same test using GROUP BY, HAVING, and NOT EXISTS?

Recall: a table-level `UNIQUE` constraint enforces a stored-data rule and rejects changes that violate it.

UNIQUE: Testing a Subquery Result

Question: does the subquery return any duplicate rows?

```
UNIQUE (SELECT course_id FROM section)
```

For `CS101, CS101, CS202`: **FALSE**, because `CS101` occurs twice.

Alternative test: useful when a DBMS does not support the UNIQUE subquery predicate.

```
NOT EXISTS (
  SELECT course_id FROM section
  GROUP BY course_id
  HAVING COUNT(*) > 1
)
```

1. **GROUP BY** groups equal course IDs.
2. **HAVING** keeps repeated IDs: `CS101`.
3. **NOT EXISTS** is FALSE: a row survives.

With `CS101, CS202`, none survives: TRUE.

Both expressions are Boolean conditions, usable after WHERE. This equivalence assumes course_id is NOT NULL.

INTERMEDIATE SQL

SECTION 04 OF 11

Subqueries as Tables and Values

Use derived tables, name intermediate results with WITH, and return scalar values.

A Subquery in FROM

Find department averages that exceed 42,000.

Schema: instructor(ID, name, dept_name, salary)

```
SELECT dept_name, avg_salary
FROM (
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor GROUP BY dept_name
) AS dept_avg
WHERE avg_salary > 42000;
```

Derived schema: dept_avg(dept_name, avg_salary)

The parenthesized query produces a derived relation, named dept_avg in this query.

WITH Names a Temporary Result

Find the department(s) with the highest budget, including ties.

Schema: `department(dept_name, building, budget)`

Now: name the maximum with WITH

```
WITH max_budget(value) AS (
  SELECT MAX(budget) FROM department
)
SELECT dept_name, budget
FROM department, max_budget
WHERE department.budget=max_budget.value;
```

CTE = Common Table Expression: a named query result defined by `WITH`, usable within this statement. Here, `max_budget` names the result and `value` its column.

dept_name	budget
Biology	90,000
Comp. Sci.	120,000
Physics	120,000

Recall: remove smaller budgets

```
SELECT dept_name, budget FROM department
WHERE budget IS NOT NULL
EXCEPT
SELECT D.dept_name, D.budget
FROM department AS D, department AS E
WHERE D.budget < E.budget;
```

Remove each department with a lower budget than another. Keep all ties for the maximum.

WITH: compute 120000, then pair each department with it and keep equal budgets.
EXCEPT: remove Biology because $90000 < 120000$.
Both output: (Comp. Sci., 120000), (Physics, 120000).

Composing Common Table Expressions

Schema: `instructor(ID, name, dept_name, salary)`

```
WITH dept_total(dept_name,value) AS (
 SELECT dept_name, SUM(salary)
 FROM instructor GROUP BY dept_name
), dept_total_avg(value) AS (
 SELECT AVG(value) FROM dept_total
)
SELECT dept_name
FROM dept_total, dept_total_avg
WHERE dept_total.value > dept_total_avg.value;
```

Example input: instructor

Name omitted; salaries in thousands.

ID	dept_name	salary
101	Biology	40
102	Biology	60
103	Comp. Sci.	90
104	Comp. Sci.	110
105	Physics	60

Later CTEs may use earlier ones: `dept_total_avg` reads the result named `dept_total`.

Think: Calculate both CTE results, then predict the department(s) returned.
What does this query find in general?

Composing CTEs: Answer and Example

Answer: departments whose total instructor salary exceeds the average of department totals.

Schema: `instructor(ID, name, dept_name, salary)`. Example amounts are in thousands.

```
WITH dept_total(dept_name,value) AS (
  SELECT dept_name, SUM(salary)
  FROM instructor GROUP BY dept_name
), dept_total_avg(value) AS (
  SELECT AVG(value) FROM dept_total
)
SELECT dept_name FROM dept_total, dept_total_avg
WHERE dept_total.value > dept_total_avg.value;
```

Input: instructor (name omitted)

ID	dept_name	salary
101	Biology	40
102	Biology	60
103	Comp. Sci.	90
104	Comp. Sci.	110
105	Physics	60

1. dept_total

dept_name	value
Biology	100
Comp. Sci.	200
Physics	60

2. dept_total_avg

value
120

$(100 + 200 + 60)/3 = 120$.
Average of **department totals**, not individual salaries.

3. Output

dept_name
Comp. Sci.

Only $200 > 120$.

Scalar Subquery in SELECT

For each department, display its name and the number of instructors in it.

Example input: department

dept_name	building	budget
Biology	Watson	90000
Physics	Watson	70000
History	Painter	50000

Example input: instructor

ID	name	dept_name	salary
101	Chen	Biology	60000
102	Lee	Biology	75000
103	Wu	Physics	80000

```
SELECT D.dept_name,
  (SELECT COUNT(*) FROM instructor AS I
   WHERE I.dept_name = D.dept_name)
 AS num_instructors
FROM department AS D;
```

Output

dept_name	num_instructors
Biology	2
Physics	1
History	0

Correlated: D.dept_name comes from the current department row; the subquery counts its matching instructors.
Scalar: COUNT(*) without GROUP BY returns one row with one value, even with no matches: History gets 0.
A scalar subquery returning more than one row would cause an error; this aggregate always supplies one value.

INTERMEDIATE SQL

SECTION 05 OF 11

Modifying Data

Delete, insert, and update rows; use CASE and subqueries to define the changes.

Data-Manipulation Statements

DELETE	INSERT	UPDATE
remove tuples	add tuples	change values

Each statement may contain a query that determines which tuples are changed or what values are written.

Deleting Tuples

ALL ROWS

```
DELETE FROM instructor;
```

ONE DEPARTMENT

```
DELETE FROM instructor
WHERE dept_name='Finance';
```

NESTED TEST

```
DELETE FROM instructor
WHERE dept_name IN (
SELECT dept_name FROM
department
WHERE building='Watson');
```

The WHERE clause selects rows from the target relation. Without WHERE, every row is selected for deletion.

Delete Below the Original Average

Schema: `instructor(ID, name, dept_name, salary)`

```
DELETE FROM instructor
WHERE salary < (SELECT AVG(salary) FROM instructor);
```

Initial salaries: Alice 30, Bob 50, Carol 60, David 100 (thousands). Original average $= 60$.

Correct: use the original average

Delete Alice and Bob: both earn less than 60.
Keep Carol and David. Carol's 60 equals the original average, so she must remain.

Wrong: recompute after each deletion

Delete Alice $\rightarrow$ average becomes 70.
Delete Bob $\rightarrow$ average becomes 80.
Then delete Carol too: $60 < 80$.

Why this is a problem: in a single pass, checking Carol first would keep her ($60 \not< 60$); checking her after Alice and Bob would delete her. The result would depend on row-processing order.

Insert Explicit Values

```
INSERT INTO course(course_id,title,dept_name,credits)
VALUES ('CS-437','Database Systems','Comp. Sci.',4);
```

Listing target columns protects the statement from schema-order assumptions and documents the meaning of each literal.

Omitted columns receive their default value or NULL; the insert fails if a required value has neither.

Insert the Result of a Query

Build a graduation review list for students with at least 120 credits (the threshold in this example).

Source: `student(ID, name, dept_name, tot_cred)`
Target: `graduation_review(ID, name, dept_name, tot_cred)` — already created and empty.

```
INSERT INTO graduation_review
 (ID, name, dept_name, tot_cred)
SELECT ID, name, dept_name, tot_cred
FROM student
WHERE tot_cred >= 120;
```

1. **SELECT** produces Alice's and Carol's rows.
2. **INSERT** copies those rows into the target; the original student rows remain unchanged.

Selected values map to target columns **by position**; their data types must be compatible.

Source rows (all in Biology)

ID	name	tot_cred
1	Alice	128
2	Bob	96
3	Carol	120

Target after insertion

ID	name	tot_cred
1	Alice	128
3	Carol	120

Both inserted rows also contain `dept_name = Biology`.

Two UPDATE Statements: Does Order Matter?

Give a 3% raise to salaries above 100,000 and a 5% raise to the rest, based on original salaries.

Schema: `instructor(ID, name, dept_name, salary)`. Run each method on the same initial data.

Method A: 3% first, then 5%

```
UPDATE instructor
SET salary = salary * 1.03
WHERE salary > 100000;

UPDATE instructor
SET salary = salary * 1.05
WHERE salary <= 100000;
```

Method B: 5% first, then 3%

```
UPDATE instructor
SET salary = salary * 1.05
WHERE salary <= 100000;

UPDATE instructor
SET salary = salary * 1.03
WHERE salary > 100000;
```

Think: Do these methods produce the same result? Trace an original salary of 98,000 through both statements in each method. Which method implements the intended policy?

Two UPDATE Statements: Answer

Method A is correct. The second statement sees the salaries left by the first statement.

Start each method independently with the same salary: **98,000**.

A: 3% first, then 5%

1. $98000 > 100000$ is FALSE: no change.
2. $98000 \leq 100000$ is TRUE:
$98000 \times 1.05 = \mathbf{102900}$.

B: 5% first, then 3%

1. $98000 \times 1.05 = 102900$.
2. Now $102900 > 100000$ is TRUE:
$102900 \times 1.03 = \mathbf{105987}$.

Why A works: salaries originally above 100,000 stay above it after a 3% raise, so the second statement skips them. The remaining rows receive only the 5% raise.

Why B fails: a 5% raise can cross the threshold, causing the same row to receive both raises.

CASE Makes the Choice Once

One statement: give each instructor either a 5% raise or a 3% raise, based on the original salary.

Schema: `instructor(ID, name, dept_name, salary)`

```
UPDATE instructor
SET salary = CASE
  WHEN salary <= 100000
  THEN salary * 1.05
  ELSE salary * 1.03
END;
```

How to read CASE

WHEN tests the row's original salary.
THEN supplies the value if TRUE.
ELSE supplies the value otherwise.
END closes the expression.

`SET` assigns that one value to `salary`.

Examples: each row takes only one branch; crossing 100,000 does not trigger another raise.

Original salary	WHEN condition	Chosen branch	New salary
98,000	TRUE	THEN: 98000×1.05	102,900
100,000	TRUE	THEN: 100000×1.05	105,000
120,000	FALSE	ELSE: 120000×1.03	123,600

UPDATE with a Scalar Subquery

Think for one minute: What does this UPDATE statement do?

Schemas: `student(ID, name, dept_name, tot_cred)`
`takes(ID, course_id, sec_id, semester, year, grade)`
`course(course_id, title, dept_name, credits)`

```
UPDATE student AS S
SET tot_cred = CASE
 WHEN (SELECT SUM(C.credits)
   FROM takes AS T JOIN course AS C
    ON T.course_id = C.course_id
   WHERE T.ID = S.ID AND T.grade <> 'F') IS NOT NULL
 THEN (SELECT SUM(C.credits)
   FROM takes AS T JOIN course AS C
    ON T.course_id = C.course_id
   WHERE T.ID = S.ID AND T.grade <> 'F')
 ELSE 0
END;
```

Example: recompute credits

`course`: BIO101 has 3 credits; BIO102 has 4.

`takes`: Alice has A in BIO101 and B in BIO102. Bob has only F or NULL grades.

Alice: SUM $= 3 + 4 = 7$.
WHEN is TRUE; **THEN** sets `tot_cred` to 7.

Bob: no qualifying rows, so SUM is NULL; **ELSE** sets 0.

Correlated: `T.ID = S.ID` selects the current student's records. **END** closes CASE; the chosen value replaces `tot_cred`. Assume non-F, non-NULL grades earn credits and no repeated courses. `grade <> 'F'` also excludes NULL (UNKNOWN).

UPDATE with a Scalar Subquery

Recompute each student's total earned credits and update `student.tot_cred`; use 0 if none qualify.

Schemas: `student(ID, name, dept_name, tot_cred)`
`takes(ID, course_id, sec_id, semester, year, grade)`
`course(course_id, title, dept_name, credits)`

```
UPDATE student AS S
SET tot_cred = CASE
 WHEN (SELECT SUM(C.credits)
   FROM takes AS T JOIN course AS C
    ON T.course_id = C.course_id
   WHERE T.ID = S.ID AND T.grade <> 'F') IS NOT NULL
 THEN (SELECT SUM(C.credits)
   FROM takes AS T JOIN course AS C
    ON T.course_id = C.course_id
   WHERE T.ID = S.ID AND T.grade <> 'F')
 ELSE 0
END;
```

Example: recompute credits

`course`: BIO101 has 3 credits; BIO102 has 4.

`takes`: Alice has A in BIO101 and B in BIO102. Bob has only F or NULL grades.

Alice: SUM $= 3 + 4 = 7$.
WHEN is TRUE; **THEN** sets `tot_cred` to 7.

Bob: no qualifying rows, so SUM is NULL; **ELSE** sets 0.

Correlated: `T.ID = S.ID` selects the current student's records. **END** closes CASE; the chosen value replaces `tot_cred`. Assume non-F, non-NULL grades earn credits and no repeated courses. `grade <> 'F'` also excludes NULL (UNKNOWN).

SECTION 06 OF 11

Join Expressions

Compare cross, inner, and outer joins; choose matching conditions and preserve unmatched rows.

Join Expressions

A join takes two input relations and returns one relation. It specifies both **which rows match** and **which columns appear**.

JOIN CONDITION

`ON`, `USING`, or `NATURAL`

JOIN TYPE

inner, left, right, or full

A DBMS may optimize a join without materializing a Cartesian product.

Join Taxonomy

Cross	Inner	Outer
every pair	matching rows	matches + preserved rows
no match predicate	equi / natural	left / right / full

A self-join is any join that uses two correlation names for the same base relation.

CROSS JOIN

EXPLICIT

```
SELECT *
FROM employee CROSS JOIN
department;
```

COMMA FORM

```
SELECT *
FROM employee, department;
```

Both forms return $|employee| \times |department|$ rows. A missing join condition is a common source of unexpectedly large results.

CROSS JOIN: Every Pair of Rows

course C = course; P = prereq

course_id	title	dept_name	credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq

course_id	prereq_id
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

```
SELECT * FROM course AS C CROSS JOIN prereq AS P;
```

Result: $3 \times 3 = 9$ **rows.** Both course_id columns remain; no equality test is applied.

C.course_id	title	dept_name	credits	P.course_id	prereq_id
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
BIO-301	Genetics	Biology	4	CS-190	CS-101
BIO-301	Genetics	Biology	4	CS-347	CS-101
CS-190	Game Design	Comp. Sci.	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101
CS-190	Game Design	Comp. Sci.	4	CS-347	CS-101
CS-315	Robotics	Comp. Sci.	3	BIO-301	BIO-101
CS-315	Robotics	Comp. Sci.	3	CS-190	CS-101
CS-315	Robotics	Comp. Sci.	3	CS-347	CS-101

INNER JOIN: ON vs. NATURAL

course C = course; P = prereq, prerequisite

course_id	title	dept_name	credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq

course_id	prereq_id
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

INNER keeps matching pairs only. **ON** states the condition explicitly; **NATURAL** infers equality on **every shared column name**. Here only `course_id` is shared, so both queries match the same two pairs.

```
SELECT * FROM course AS C
INNER JOIN prereq AS P
ON C.course_id = P.course_id;
```

ON result: both ID columns remain

C.course_id	title	dept.	cr.	P.course_id	prereq_id
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101

```
SELECT * FROM course
NATURAL INNER JOIN prereq;
```

No ON clause: names determine the match.

NATURAL result: shared ID column appears once

course_id	title	dept.	cr.	prereq_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

Equi-Join versus Natural Join

EQUI-JOIN WITH ON

```
course INNER JOIN prereq ON
course.course_id=prereq.course_id
```

Keeps both join columns.

NATURAL JOIN

```
course NATURAL INNER JOIN prereq
```

Matches every shared name and outputs each once.

Natural joins can change meaning after a schema adds another same-named column; `ON` or `USING` makes the condition explicit.

Outer Joins: Preserving Unmatched Rows

An inner join keeps only matching pairs. An **outer join** additionally preserves unmatched rows from one or both inputs, filling the missing side's attributes with **NULL**.

```
L {LEFT | RIGHT | FULL} OUTER JOIN R ON condition
```

LEFT OUTER JOIN

Preserve every row of L.
For an unmatched L row,
fill R's attributes with NULL.

RIGHT OUTER JOIN

Preserve every row of R.
For an unmatched R row,
fill L's attributes with NULL.

FULL OUTER JOIN

Preserve every row of both.
For each unmatched row,
fill the other side with NULL.

A pair matches when the condition is TRUE. A row with multiple partners produces multiple result rows.
Left/right means position in the SQL expression. Choose one join type above; the keyword OUTER is optional.

LEFT OUTER JOIN: Two Result Shapes

course C = course; P = prereq

course_id	title	dept_name	credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq

course_id	prereq_id
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

Same matched and preserved rows; different output columns with SELECT *.

```
SELECT * FROM course AS C
LEFT OUTER JOIN prereq AS P
ON C.course_id = P.course_id;
```

ON: two course_id columns.

C.course_id	title	dept_name	credits	P.course_id	prereq_id
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101
CS-315	Robotics	Comp. Sci.	3	*NULL*	*NULL*

```
SELECT * FROM course
NATURAL LEFT OUTER JOIN prereq;
```

NATURAL: one course_id column.

course_id	title	dept_name	credits	prereq_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*NULL*

RIGHT OUTER JOIN: Two Result Shapes

course C = course; P = prereq

course_id	title	dept_name	credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq

course_id	prereq_id
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

Same matched and preserved rows; different output columns with SELECT *.

```
SELECT * FROM course AS C
RIGHT OUTER JOIN prereq AS P
ON C.course_id = P.course_id;
```

ON: two course_id columns.

C.course_id	title	dept_name	credits	P.course_id	prereq_id
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101
NULL	*NULL*	*NULL*	*NULL*	CS-347	CS-101

```
SELECT * FROM course
NATURAL RIGHT OUTER JOIN prereq;
```

NATURAL: one course_id column.

course_id	title	dept_name	credits	prereq_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	*NULL*	*NULL*	*NULL*	CS-101

FULL OUTER JOIN: Two Result Shapes

course C = course; P = prereq

course_id	title	dept_name	credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq

course_id	prereq_id
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

Same matched and preserved rows below; different output columns with SELECT *.

```
SELECT * FROM course AS C
FULL OUTER JOIN prereq AS P
ON C.course_id = P.course_id;
```

ON: two course_id columns.

C.course_id	title	dept_name	credits	P.course_id	prereq_id
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101
CS-315	Robotics	Comp. Sci.	3	*NULL*	*NULL*
NULL	*NULL*	*NULL*	*NULL*	CS-347	CS-101

```
SELECT * FROM course
NATURAL FULL OUTER JOIN prereq;
```

NATURAL: one course_id column.

course_id	title	dept_name	credits	prereq_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*NULL*
CS-347	*NULL*	*NULL*	*NULL*	CS-101

Think: Is a FULL OUTER JOIN equivalent to a Cartesian product? Why or why not?

USING: Name the Columns to Match

course C = course; P = prereq

course_id	title	dept_name	credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq

course_id	prereq_id
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

```
SELECT * FROM course FULL OUTER JOIN prereq USING (course_id);
```

Match condition: `course.course_id = prereq.course_id`.
The named column must exist in **both** input tables.
FULL keeps unmatched rows from both sides.
USING outputs the join column **once**; for an unmatched row, its ID comes from the preserved side.

course_id	title	dept.	cr.	prereq_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*NULL*
CS-347	*NULL*	*NULL*	*NULL*	CS-101

USING: equality on *only the listed columns*; `USING(a,b)` means both `L.a=R.a` and `L.b=R.b`.
NATURAL: equality on *all shared names*. **ON:** write the condition yourself; join columns remain separate.
If both tables later add `year`, NATURAL also compares year; `USING(course_id)` still compares only the ID.

Join Condition and Join Type Are Independent

Join type	Preserves
`INNER`	matches only
`LEFT OUTER`	left unmatched
`RIGHT OUTER`	right unmatched
`FULL OUTER`	both unmatched

Condition	How matched
`NATURAL`	all shared names
`USING(A)`	listed shared names
`ON(P)`	arbitrary predicate

Choose one item from each column: for example, `LEFT OUTER JOIN ... ON ....`

SECTION 07 OF 11

Defining and Expanding Views

Name reusable queries and follow their definitions and dependencies.

Views: Definition and Purpose

A **view** is a **named query** that you can use **like a table**. An ordinary view stores the query definition, not a separate copy of its rows; querying it reads the underlying data visible to that statement.

Syntax: `CREATE VIEW v AS <query expression>;`

Base table: `instructor(ID, name, dept_name, salary)`

```
CREATE VIEW faculty AS
SELECT ID, name, dept_name
FROM instructor;
```

Query it like a table:

```
SELECT name FROM faculty
WHERE dept_name = 'Biology';
```

The result lists Biology instructors' names.

View schema: `faculty(ID, name, dept_name)`. Salary is omitted; changed instructor data is reflected when queried.

Security
Expose selected columns. Grant access to the view while restricting the base table.

Simplicity
Write a complex join once in a view; later queries use its name and add filters.

Stability
Adapt the view when base tables change, so application queries can stay unchanged.

Example Views

HIDE SALARY

```
CREATE VIEW faculty AS
SELECT ID,name,dept_name
FROM instructor;

SELECT name FROM faculty
WHERE dept_name='Biology';
```

DEPARTMENT TOTALS

```
CREATE VIEW departments_total_salary
(dept_name,total_salary) AS
SELECT dept_name,SUM(salary)
FROM instructor GROUP BY dept_name;
```

The view column list gives stable, meaningful names to computed columns such as `SUM(salary)`.

A View Defined Using Another View

Base schemas:
course(course_id, title, dept_name, credits)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

```
CREATE VIEW physics_fall_2009 AS
SELECT course.course_id,sec_id,building,room_number
FROM course,section
WHERE course.course_id=section.course_id
AND course.dept_name='Physics'
AND section.semester='Fall'
AND section.year=2009;
```



```
CREATE VIEW physics_fall_2009_watson AS
SELECT course_id,room_number
FROM physics_fall_2009
WHERE building='Watson';
```

View schemas:
physics_fall_2009(course_id, sec_id, building, room_number)
physics_fall_2009_watson(course_id, room_number)

The second view reads the first view and keeps only sections in the Watson building.

View Expansion

Expansion replaces a view name with the query that defines it.



After expansion, the combined query applies all **five predicates** to the base relations.

Join: `course.course_id = section.course_id`
Filters: `course.dept_name='Physics' AND section.semester='Fall'`
`AND section.year=2009 AND section.building='Watson'`

View Expansion and Dependencies

Expanding a view follows its dependencies: the tables or views used to compute its result.

How to expand

01 Find a view reference in the query.

02 Replace it with its defining query.

03 Repeat until only base tables remain.

When does it finish?

Direct dependency
A view references another in its definition.

Indirect dependency
The dependency passes through other views.

Recursive dependency
A chain leads back to the same view.

No cycle: expansion finishes. **Cycle:** repeated substitution cannot finish; recursion needs special semantics and support.

SECTION 08 OF 11

Updating and Materializing Views

Determine when a view can be updated and when its results are stored.

Updating a Simple View

Base table: `instructor(ID, name, dept_name, salary)`
View definition: `CREATE VIEW faculty AS SELECT ID, name, dept_name FROM instructor;`

1. You insert through the view

```
INSERT INTO faculty (ID, name, dept_name) VALUES ('30765', 'Green', 'Music');
```

2. The DBMS writes the corresponding base-table row

```
INSERT INTO instructor (ID, name, dept_name) VALUES ('30765', 'Green', 'Music');
```

What gets stored?

The view stores no rows; its columns map to instructor.
Salary uses its default. If none exists and NULL is allowed:

`('30765', 'Green', 'Music', NULL)`

The view shows only ID, name, and dept_name.

When can the insert fail?

The DBMS must support inserts through this view, and all base-table constraints must hold.

A NOT NULL salary with no default causes failure; so does an ID that duplicates an existing primary key.

Join-View Inserts: An Ambiguous Mapping

Two departments share Taylor. The view shows the building, but hides the department name.

instructor

ID	name	dept_name	salary
101	Chen	Comp. Sci.	80000
102	Lee	Elec. Eng.	75000

department

dept_name	building	budget
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000

instructor_info

ID	name	building
101	Chen	Taylor
102	Lee	Taylor

```
CREATE VIEW instructor_info AS
SELECT I.ID, I.name, D.building
FROM instructor AS I JOIN department AS D
ON I.dept_name = D.dept_name;
```

Execution in PostgreSQL

Without an INSTEAD OF trigger or rule, this join-view INSERT raises an error. **No row is added.**

Assume: ID 69987 is new, and salary allows NULL with no default.

```
INSERT INTO instructor_info VALUES ('69987','White','Taylor');
```

Alternatives, not executed

ID	name	dept_name	salary
69987	White	Comp. Sci.	*NULL*
69987	White	Elec. Eng.	*NULL*

Either alternative yields the requested view row; the query does not choose a department. Other DBMSs have their own rules; an explicit trigger can define the mapping. (These are not two inserted rows.)

View Inserts: What Does `CHECK OPTION` Do?

Schema: `instructor(ID, name, dept_name, salary)`. The view below shows only History instructors.

Assume view inserts are supported, ID 25566 is new, and all base-table constraints hold.

A. Without CHECK OPTION

```
CREATE VIEW history_instructors AS
SELECT * FROM instructor
WHERE dept_name = 'History';
```

B. Append it after the view query

```
CREATE VIEW history_instructors AS
SELECT * FROM instructor
WHERE dept_name = 'History'
WITH CHECK OPTION;
```

Try the same insert in each case:

```
INSERT INTO history_instructors VALUES ('25566','Brown','Biology',100000);
```

Outcome for Brown	A. No CHECK OPTION	B. WITH CHECK OPTION
INSERT	Succeeds under assumptions below	Rejected: Biology is not History
Base table instructor	Brown is stored	No Brown row is inserted
View history_instructors	Brown is filtered out (0 rows)	No Brown row (insert failed)

Effect: INSERT/UPDATE through this view must leave rows satisfying `dept_name='History'`.
For example, changing a visible instructor's department to Biology is rejected; keeping History passes this check.

Materialized Views: Store, Read, Refresh

A materialized view stores the **computed rows**, not just the query definition. Reads reuse that stored result.

Base table: instructor (salaries in thousands)

ID	name	dept_name	salary
101	Chen	Biology	40
102	Lee	Biology	60

1. Create (PostgreSQL syntax)

```
CREATE MATERIALIZED VIEW dept_payroll AS
SELECT dept_name, SUM(salary) AS total
FROM instructor GROUP BY dept_name;
```

2. Change the base table

```
UPDATE instructor SET salary = 80
WHERE ID = '102';
```

3. Refresh

```
REFRESH MATERIALIZED VIEW dept_payroll;
```

Stored result: dept_payroll

dept_name	total
Biology	100

What does SELECT * FROM dept_payroll return?

After step	dept_name	total
1. Create	Biology	100
2. UPDATE	Biology	100
3. REFRESH	Biology	120

After step 2, salaries are 40 and 80, but the stored total is still 100. PostgreSQL does not refresh it automatically.

Trade-off: faster repeated reads versus storage, refresh work, and stale results. Refresh policies vary by DBMS.

INTERMEDIATE SQL

SECTION 09 OF 11

Transactions and Constraints

Group changes into transactions and enforce valid rows and references.

Transactions: Start, Commit, or Roll Back

A transaction groups statements into **one logical unit of work**. Atomicity means its changes are committed together or undone together, rather than leaving only part of the work completed.

BEGIN

Start a transaction. Subsequent statements belong to this unit until it ends.

COMMIT

End successfully and make the changes durable. A later ROLLBACK cannot undo them.

ROLLBACK

End without committing. Undo this transaction's changes to transactional data.

Choose one ending: COMMIT accepts the work; ROLLBACK cancels the uncommitted work. For example, a transfer must both debit the sender and credit the recipient.

Transaction-start syntax and autocommit behavior vary by DBMS and client. The next example uses PostgreSQL-style BEGIN and transactional tables.

A Transaction in SQL: Transfer 100

Schema: `account(id, balance)`. A and B exist, IDs are unique, and there are no concurrent changes.

SQL SENT ON THE SUCCESS PATH

```
BEGIN;
UPDATE account
SET balance = balance - 100
WHERE id = 'A';
UPDATE account
SET balance = balance + 100
WHERE id = 'B';
COMMIT;
```

id	Initially	Commit	Rollback
A	1000	900	1000
B	500	600	500

One transaction, one ending

`BEGIN` starts the transaction. Both UPDATEs belong to it.

If both succeed, the application sends `COMMIT`.

If an UPDATE fails, the application's error handler sends `ROLLBACK` instead of `COMMIT`.

The application chooses the ending.
SQL does not automatically branch from this script to ROLLBACK.

Integrity Constraints and Candidate Keys

Integrity constraints protect data consistency: even an authorized statement is rejected if it violates a constraint.

FOUR COMMON CONSTRAINTS

NOT NULL A value is required.

PRIMARY KEY The chosen identifier: unique and non-null.

UNIQUE No duplicate non-null keys.

CHECK A predicate must not be false.

SYNTAX AND KEY SEMANTICS

```
name varchar(20) NOT NULL
budget numeric(12,2) NOT NULL

UNIQUE (A1, A2, ..., Am)

ID varchar(5) PRIMARY KEY
```

UNIQUE alone is not a candidate key.
It may allow NULLs; their treatment varies by DBMS. It does not require minimality either.

For a candidate key, require non-null values and a minimal unique attribute set.

`UNIQUE` + `NOT NULL` on every key column + minimality ⇒ candidate key.

The CHECK Clause

`CHECK(P)` tests each row: the predicate P must **not be FALSE**.

EXAMPLE: PERMITTED SEMESTER VALUES

```
CREATE TABLE section (
  course_id varchar(8),
  sec_id varchar(8),
  semester varchar(6),
  year numeric(4,0),
  CHECK (semester IN
    ('Fall', 'Winter', 'Spring', 'Summer'))
);
```

TWO INSERTS INTO THIS TABLE

```
INSERT INTO section VALUES
('CS-101', '1', 'Fall', 2026);
```

TRUE → accepted.
The semester is in the allowed list.

```
INSERT INTO section VALUES
('CS-102', '1', NULL, 2026);
```

UNKNOWN → accepted.
CHECK permits NULL in this example.

A value is required? Add NOT NULL.
A NULL semester makes this predicate UNKNOWN, so this CHECK alone does not reject it.

Referential Integrity: Who References Whom?

Each non-null `course.dept_name` must match an existing `department.dept_name`.

Schemas: `course(course_id, title, dept_name, credits)`
`department(dept_name, building, budget)`

DECLARED INSIDE CREATE TABLE COURSE

```
FOREIGN KEY (dept_name)
REFERENCES department(dept_name)
```

course is the referencing table.
Many courses may share the same department; its foreign-key values need not be unique.

department is the referenced table.
Its referenced column needs a PRIMARY KEY or UNIQUE constraint.

Add `NOT NULL` to `course.dept_name` if every course must belong to a department.

Existing department rows

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000

A new course's dept_name

Value	Foreign-key check
Biology	Pass: a matching row exists
History	Fail: no matching row
NULL	Pass if NULL is allowed

Cascading Deletes and Updates

CASCADE preserves referential integrity by propagating parent changes to matching child rows.
DELETE: delete those child rows. **UPDATE:** copy the new referenced-key value into their foreign keys.

Schemas: `department(dept_name, building, budget)`; `course(course_id, title, dept_name, credits)`.

```
FOREIGN KEY (dept_name) REFERENCES department(dept_name) ON DELETE CASCADE ON UPDATE CASCADE
```

Initial state

Each operation starts here.
Only key columns are shown;
other surviving values stay unchanged.

department

dept_name
Biology
Comp. Sci.

course

course_id	dept_name
BIO-101	Biology
BIO-301	Biology
CS-101	Comp. Sci.

Delete Biology

```
DELETE FROM department
WHERE dept_name = 'Biology';
```

department

dept_name
Comp. Sci.

course

course_id	dept_name
CS-101	Comp. Sci.

Rename Biology to Bio

```
UPDATE department
SET dept_name = 'Bio'
WHERE dept_name = 'Biology';
```

department

dept_name
Bio
Comp. Sci.

course

course_id	dept_name
BIO-101	Bio
BIO-301	Bio
CS-101	Comp. Sci.

Think for 1 minute: From the initial tables, rename 'Biology' to 'Life Sci.', then delete 'Life Sci.'. Write both SQL statements and show the final contents of both tables.

Answer to the last question

Start from the original tables. Rename `Biology` to `Life Sci.`, then delete that department.
Both ON UPDATE CASCADE and ON DELETE CASCADE are enabled on the course foreign key.

Initially: department

dept_name
Biology
Comp. Sci.

Initially: course

course_id	dept_name
BIO-101	Biology
BIO-301	Biology
CS-101	Comp. Sci.

```
UPDATE department SET dept_name = 'Life Sci.'
WHERE dept_name = 'Biology';

DELETE FROM department
WHERE dept_name = 'Life Sci.';
```

After UPDATE: both BIO courses reference Life Sci.
After DELETE: that department and both BIO course rows are removed.

Finally: department

dept_name
Comp. Sci.

Finally: course

course_id	dept_name
CS-101	Comp. Sci.

Other actions: `NO ACTION`, `RESTRICT`, `SET NULL`, `SET DEFAULT`; support and timing vary by DBMS.

Foreign-Key Timing: Why Does This Insert Fail?

Rule: a non-null `person.mother` must match an existing `person.ID`.
We want to insert Alice and her mother Mary, starting with an empty `person` table.

```
CREATE TABLE person (
 ID integer PRIMARY KEY,
 name varchar(20), mother integer,
 CONSTRAINT mother_reference
  FOREIGN KEY (mother) REFERENCES person(ID)
  DEFERRABLE INITIALLY IMMEDIATE
);
```

INITIALLY IMMEDIATE: check after each statement by default.

DEFERRABLE: the transaction *may* postpone this check; it does not do so automatically.

The data we want to store

ID	name	mother
1	Alice	2
2	Mary	NULL

Alice's mother is Mary (ID 2).

Try inserting Alice first:

```
INSERT INTO person VALUES
(1, 'Alice', 2);
```

Fails at the end of this statement.
ID 2 is missing, so Alice's row is not stored. Insert Mary first, or defer the check.

Deferred Checking: Fix the Reference Before COMMIT

Same table: `person(ID, name, mother)`, initially empty; `mother` references `ID`.
The named constraint `mother_reference` was declared `DEFERRABLE`.

PostgreSQL syntax. SET CONSTRAINTS mother_reference IMMEDIATE would instead force the pending check at that point.

```
BEGIN;
SET CONSTRAINTS mother_reference DEFERRED;

INSERT INTO person VALUES (1, 'Alice', 2);

INSERT INTO person VALUES (2, 'Mary', NULL);

COMMIT;
```

DEFERRED postpones this foreign-key check
until COMMIT in this transaction.

If Mary's INSERT is omitted, COMMIT fails:
an unresolved reference cannot be committed.

After the first INSERT (uncommitted)

ID	name	mother
1	Alice	2

ID 2 is missing, but the check is postponed.

After the second INSERT

ID	name	mother
1	Alice	2
2	Mary	NULL

COMMIT checks the reference: ID 2 exists.
The transaction succeeds; both rows are saved.

SECTION 10 OF 11

Types and Indexes

Choose temporal and large-object types, define domains, and introduce indexes.

Built-in Temporal Types

Type	Example literal	Meaning
DATE	DATE '2005-07-27'	calendar date
TIME	TIME '09:00:30.75'	time of day
TIMESTAMP	TIMESTAMP '2005-07-27 09:00:30.75'	date and time
INTERVAL	INTERVAL '1' DAY	duration

Subtracting compatible date/time values yields an interval; adding an interval shifts a date/time value. Exact literal syntax can vary.

User-Defined Types: Give Values a Meaning

Why? A number such as `100.00` could mean money, distance, or weight.
A **distinct type** is a user-defined type that gives an existing data type a separate identity.

SQL-standard-style distinct-type syntax is shown; support, conversion rules, and exact syntax vary by DBMS.

1. DEFINE A TYPE FOR DOLLAR AMOUNTS

```
CREATE TYPE Dollars AS
NUMERIC(12,2) FINAL;
```

Dollars: the new type's name.
NUMERIC(12,2): 12 digits in total, including 2 after the decimal point.
FINAL: no subtypes can be defined.

A separate type, not just a new name.
Strong typing prevents accidental mixing; explicit conversion may be required.

2. USE IT AS A COLUMN TYPE

```
CREATE TABLE department (
 dept_name varchar(20),
 building varchar(15),
 budget Dollars
);

INSERT INTO department VALUES
('Biology', 'Watson', CAST(90000 AS Dollars));
```

dept_name	building	budget (Dollars)
Biology	Watson	90000.00

CAST changes the type to Dollars.
It does not convert between currencies.

Domains: Reuse Rules for Valid Values

A **domain** packages a base type with reusable constraints and, optionally, a default.
Use its name as a column type: each column using it inherits the same value rules.

1. DEFINE THE RULES ONCE

```
CREATE DOMAIN person_name
AS varchar(20) NOT NULL;

CREATE DOMAIN degree_level AS varchar(10)
CHECK (VALUE IN
 ('Bachelors', 'Masters', 'Doctorate'));
```

VALUE means the value being tested.
For 'Diploma', the CHECK is FALSE: reject it.
This degree domain permits NULL; add NOT NULL if a degree is required.

2. USE THE DOMAINS AND INSERT VALUES

```
CREATE TABLE applicant (
 name person_name, degree degree_level);
```

```
INSERT INTO applicant VALUES ('Lee', 'Masters');
```
Accepted: stores the row (Lee, Masters).

```
INSERT INTO applicant VALUES ('Kim', 'Diploma');
```
Rejected: degree fails the domain CHECK.

```
INSERT INTO applicant VALUES (NULL, 'Masters');
```
Rejected: name violates the domain NOT NULL.

Previous slide: distinct type
Separate type identity: avoid mixing meanings.

Domain: constrained base values
Reuse validation; retain base-type operations.

BLOB and CLOB: Binary Files vs. Long Text

BLOB = Binary Large Object
Stores bytes: photos, PDF files, audio.

CLOB = Character Large Object
Stores characters: reports, articles, long notes.

Oracle SQL examples. Small values keep the examples readable; real applications can store much larger content.

```
CREATE TABLE attachment (
 ID integer PRIMARY KEY, content BLOB);

INSERT INTO attachment VALUES
 (1, TO_BLOB(HEXTORAW('414243')));

SELECT DBMS_LOB.GETLENGTH(content)
 AS byte_count
FROM attachment WHERE ID = 1;
```

Result: byte_count = 3.
HEXTORAW decodes three bytes: 41, 42, 43 (hex). BLOB stores bytes without treating them as text.

```
CREATE TABLE report (
 ID integer PRIMARY KEY, body CLOB);

INSERT INTO report VALUES
 (1, TO_CLOB('Database notes'));

SELECT DBMS_LOB.GETLENGTH(body)
 AS character_count
FROM report WHERE ID = 1;
```

Result: character_count = 14.
The CLOB stores the text Database notes, including its space. Character count is not byte count.

For real files or long documents, applications typically stream data through a driver or LOB API instead of using huge SQL literals.

Indexes: Find Rows Without Reading Every Row

An **index** helps the database find the rows it needs without checking every row in the table.
It works like a book's index: look up a value, then go to the matching rows.

Schema: `student(ID, name, dept_name, tot_cred)`. Index `dept_name` to find students by department.

```
CREATE INDEX student_department_index
ON student(dept_name);
```

```
SELECT * FROM student
WHERE dept_name = 'Biology';
```

student: example rows

ID	name	dept_name	tot_cred
101	Chen	Comp. Sci.	60
102	Lee	Biology	45
103	Kim	Comp. Sci.	72

Lookup for Biology
Index entry → row for student 102
→ return `(102, Lee, Biology, 45)`.

Without an index: scan and test each row.
With this index: locate matching rows directly, if the optimizer chooses it.

Trade-off: indexes need extra storage and maintenance on INSERT, DELETE, and indexed-column UPDATE.
Why sometimes still choose a table scan? If the table is small or most rows match, reading it directly may cost less than searching the index and then fetching many rows. The optimizer compares estimated costs; the result is unchanged.

Primary keys commonly get a unique index automatically. An additional identical index on ID is usually redundant.

INTERMEDIATE SQL

SECTION 11 OF 11

Privileges and Roles

Grant and revoke access; organize permissions with roles and views.

Forms of Authorization

DATA PRIVILEGES

Privilege	Allows
SELECT	read rows
INSERT	add rows
UPDATE	change values
DELETE	remove rows

SCHEMA PRIVILEGES

Privilege	Allows
REFERENCES	create foreign keys
TRIGGER	create triggers
USAGE	use schema objects
CREATE	create objects

Privilege names and schema-level capabilities differ across DBMS products.

GRANT: Who May Do What, on Which Object?

GRANT gives permission to perform an operation.

```
GRANT <privileges> ON <object> TO <users or roles>;
```

Privileges: allowed operations. **Object**: the table or view. **TO**: who receives permission.

Example schema: `instructor(ID, name, dept_name, salary)`. Users and roles below already exist.

SQL statement	Permission it adds
`GRANT SELECT ON instructor TO U1,U2,U3;`	U1, U2, U3 may read instructor rows.
`GRANT INSERT ON instructor TO registrar;`	registrar may add instructor rows.
`GRANT UPDATE(salary) ON instructor TO chair;`	chair may change salary; no other column is granted here.
`GRANT DELETE ON instructor TO admin;`	admin may delete instructor rows.

Think for 1 minute: A payroll assistant must read instructor records and update only salaries.
They must not change names or departments, insert rows, or delete rows. Write the required GRANT statements.

Answer to the last question

Requirement: the payroll assistant can read instructor records and change only salary values.
Schema: `instructor(ID, name, dept_name, salary)`. Assume no other access grants.

```
GRANT SELECT ON instructor TO payroll_assistant;
GRANT UPDATE(salary) ON instructor TO payroll_assistant;
```

Allowed

```
SELECT * FROM instructor;
UPDATE instructor SET salary = 85000
WHERE ID = '101';
```

Not granted

UPDATE of name, dept_name, or ID;
INSERT of new rows; DELETE of existing rows.
Do not use ALL PRIVILEGES for this task.

Who can grant? The object **owner** or someone **with the required grant authority**.
PUBLIC as recipient means all users.

ALL PRIVILEGES: all applicable privileges the grantor can grant on that object.
A grant on a view does not grant direct access to its base tables.

REVOKE: Remove Permission

REVOKE removes a previously granted permission.

Schema: `instructor(ID, name, dept_name, salary)`. Alice and Bob are ordinary users, not owners.
Run the cases independently; assume no other grants or roles provide access. Commands shown use PostgreSQL.

1. REMOVE ALICE'S DIRECT ACCESS

Owner runs:

```
GRANT SELECT ON instructor TO Alice;
```

Alice runs:

```
SELECT * FROM instructor; -- succeeds
```

Owner then runs:

```
REVOKE SELECT ON instructor FROM Alice;
```

Alice repeats the same SELECT:
Permission denied; the query cannot read the table. The instructor rows remain unchanged.

2. ALICE HAS PASSED ACCESS TO BOB

Owner runs:

```
GRANT SELECT ON instructor TO Alice
WITH GRANT OPTION;
```

Alice may now grant SELECT to Bob:

```
GRANT SELECT ON instructor TO Bob;
```

Owner chooses one:

```
REVOKE SELECT ON instructor
FROM Alice CASCADE;
```

Removes Alice's grant and Bob's dependent grant.

```
REVOKE SELECT ON instructor
FROM Alice RESTRICT;
```

Rejected: Bob depends on Alice's grant.
Nothing is revoked: both can still SELECT.

Another independent grant (including through a role or PUBLIC) may still allow access: REVOKE removes a grant, not all possible access paths.

Roles: Share Permissions Through Membership

A **role** is a named **group of permissions**. Give a user membership in the role to **share its permissions**.
A role can also be a member of another role, so permissions can be **inherited** through several levels.

Schema: `takes(ID, course_id, sec_id, semester, year, grade)`. The login `faculty_member` already exists.
PostgreSQL example: inheritance is enabled on both memberships; the user has no other table privileges and is not the owner.

1. Give SELECT to a role

```
CREATE ROLE teaching_assistant;
GRANT SELECT ON takes TO teaching_assistant;
```

2. Let instructor inherit that role

```
CREATE ROLE instructor;
GRANT teaching_assistant TO instructor;
```

3. Give the user membership in instructor

```
GRANT instructor TO faculty_member;
```

Read GRANT A TO B as: B becomes a member of A.
`GRANT SELECT ON takes ...` grants an operation;
`GRANT instructor TO ...` grants role membership.

HOW SELECT PERMISSION REACHES THE USER

`teaching_assistant`
↓ SELECT is inherited by
`instructor`
↓ SELECT is inherited by
`faculty_member`

As faculty_member:

```
SELECT * FROM takes;
```

Succeeds: SELECT is inherited through both memberships.

`DELETE FROM takes;` fails: this role chain provides no DELETE permission.

View Authorization: Show Only the Allowed Rows

Grant SELECT on the view, not on its base table. The view's query determines which rows are returned.

```
CREATE VIEW geo_instructor AS
SELECT * FROM instructor
WHERE dept_name = 'Geology';
GRANT SELECT ON geo_instructor TO geo_staff;
```

Assume: the user inherits geo_staff's privileges and has no base-table access.
PostgreSQL's default view mode checks base-table privileges using the view owner, who has SELECT on instructor.

Base table: instructor

ID	name	dept_name	salary
101	Chen	Geology	80000
102	Lee	Physics	90000
103	Kim	Geology	75000

`SELECT * FROM instructor;`
Permission denied. No direct access to the base table, even with a Geology WHERE condition.

SELECT * FROM geo_instructor;

ID	name	dept_name	salary
101	Chen	Geology	80000
103	Kim	Geology	75000

Visible: both Geology rows, including salaries (the view uses SELECT *).
Not returned: Lee's Physics row.
No INSERT, UPDATE, or DELETE is granted.

Think for 1 minute: Let geo_staff see only Geology instructors' IDs and names, never salaries.
Define a view, grant access, and show its result using the example data. Assume no other access grants.

Answer to the last question

Goal: geo_staff sees only the IDs and names of Geology instructors.
Use the original instructor data; this is an independent setup with no other access grants.

```
CREATE VIEW geo_directory AS
SELECT ID, name FROM instructor
WHERE dept_name = 'Geology';
GRANT SELECT ON geo_directory TO geo_staff;
```

SELECT * FROM geo_directory;

ID	name
101	Chen
103	Kim

Original instructor

ID	name	dept_name	salary
101	Chen	Geology	80000
102	Lee	Physics	90000
103	Kim	Geology	75000

WHERE limits rows: no Physics row.
SELECT limits columns: no salary.

Querying instructor directly is denied. Salary is not a column of geo_directory.

In an existing setup, earlier grants on geo_instructor or instructor must also be removed; creating a restricted view does not revoke them.

REFERENCES vs. WITH GRANT OPTION

REFERENCES permits a foreign key. **WITH GRANT OPTION** permits passing a privilege to someone else.

Existing table: `department(dept_name, building, budget)`; `dept_name` is its primary key.
Example row: `('Biology', 'Watson', 90000)`. Assume no other grants; the designer can create tables in a schema.

1. PERMISSION TO REFERENCE A KEY

Department owner runs:

```
GRANT REFERENCES(dept_name)
ON department TO schema_designer;
```

Schema_designer may then run:

```
CREATE TABLE course (
 course_id varchar(8) PRIMARY KEY,
 dept_name varchar(20)
  REFERENCES department(dept_name));
```

The foreign key can be created.
`SELECT * FROM department;` still fails:
REFERENCES does not grant SELECT.

2. PERMISSION TO PASS SELECT ONWARD

Department owner runs:

```
GRANT SELECT ON department
TO manager WITH GRANT OPTION;
```

Manager may then run:

```
GRANT SELECT ON department TO analyst;
```

Analyst can now SELECT department rows.
Without the grant option, manager could read the table but could not pass SELECT to analyst.

Revoking manager's grant: CASCADE removes the dependent grant; RESTRICT refuses.

Lecture Summary

01 Nested queries express membership, quantified comparison, existence, derived relations, and scalar values.

02 Data-modification statements combine queries with atomic row changes.

03 Join type determines preserved rows; views name queries and may or may not be updatable.

04 Transactions and constraints protect consistency; types, indexes, and privileges shape use and access.